The problem: airport answers are scattered

Where answers live today

Flight status
In airline apps and flight data APIs

SOPs and complaints
In PDF and Word documents

Aerodrome and concession info
Spread across website pages

Result: people search several places, call, or visit in person.

Who it is for

Passengers and visitors

Flight status, lost baggage, complaints

ATC applicants

How to register for the ATC exam

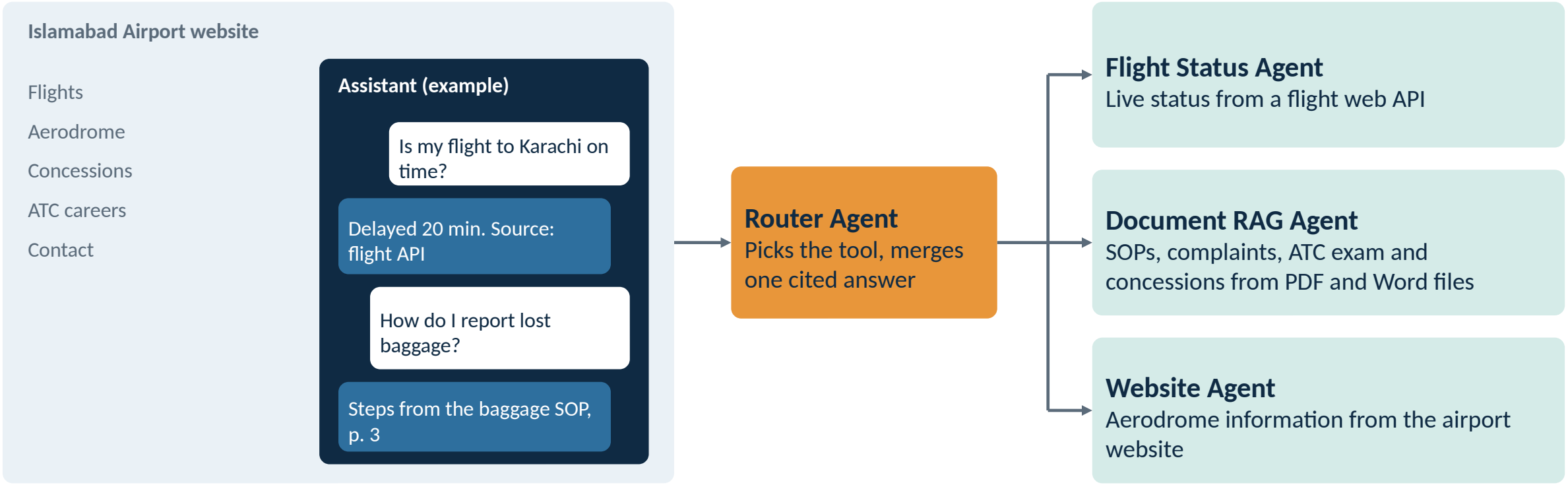
Businesses

Concessionary spaces and how to apply

Staff and stakeholders

SOPs and aerodrome information

The solution: one chatbot, three specialist agents



Check flight status in the chat

Get SOP and complaint guidance

Ask about ATC exams and concessions

Find aerodrome information

Approach: technologies and data sources

Component	Expected technology	Data source
Chat interface	Streamlit chat, shown beside the airport website	Islamabad Airport website
Router and agents	LLM with tool calling (LangGraph or similar)	Hosted LLM API (to be chosen)
Document RAG	Chunking, embeddings, ChromaDB vector store	SOPs, forms and guides in PDF and Word (public or sanitized)
Flight status	Web API client with caching and timeouts	Flight API such as AviationStack or AeroDataBox (to be tested)
Website agent	Fetch and clean pages, index text with dates	Airport website pages
Guardrails and tests	Scope check, injection defence, citations	50 test questions and 10 red-team prompts

All libraries, APIs, models and data used will be cited in the README.

Scope and risks

In scope

- Streamlit chat beside the airport website
- Router and three agents
- Cited, grounded answers
- Basic guardrails and a test set

Out of scope

- Ticket booking and payments
- User accounts and personal data
- Live links to internal airport systems
- Mobile app

Risks and how I will reduce them

Flight API gaps or rate limits
Test early, cache results, use a mock feed as a demo fallback

Website changes or blocks fetching
Keep a dated snapshot of the pages

Wrong or invented answers
Answer only from retrieved text, cite sources, say "I don't know"

Confidential SOPs
Use public or sanitized documents only, mask personal data

Misrouted questions
Test on 50 questions, ask a clarifying question when unsure

Plan and success criteria

Milestones

1 Feasibility

Test the flight API, collect documents, fetch website pages

2 Core agents

Document RAG, flight agent, website agent and router

Midpoint check-in

3 Chat and guardrails

Streamlit chat beside the website, scope and injection checks

4 Test and polish

Test set, red-team prompts, README, final demo

Final

How I will show it works (targets)

90%+

of 50 test questions routed to the right agent

100%

of flight answers match the API response

85%+

of document answers supported by the cited source

10 / 10

red-team prompts (off-topic, injection) handled safely

Live demo: flight status, lost baggage, ATC exam registration, concession space, aerodrome information